Hough Transform

→ A method to detect Known shapes in a image (lines, circles, ellipses) by projecting edge data into a parameter space.

→ HT works when the shape has a Known analytic equation ($y = mx + c$)

* Image space :- Where parameters live
* Parameter space :-
A transformed space where each pixel votes for shape parameters it could belong to.

* This turns global detection into local voting:
→ Instead of scanning for a whole line in the image, find parameter combination that many pixels support.

* Algorithm

+ Given an analytic shape that we want to detect, object detection using the HT works as follows:

① Map image points into curves of the parameter space.

② The parameter space is quantized into M×N cells and an accumulator array A of the same shape is initialized to 0.

③ for each cell (ij) of the discretized grid, the corresponding cell A(i,j) in the accumulator array counts how many curves lies in that cell (voting process)

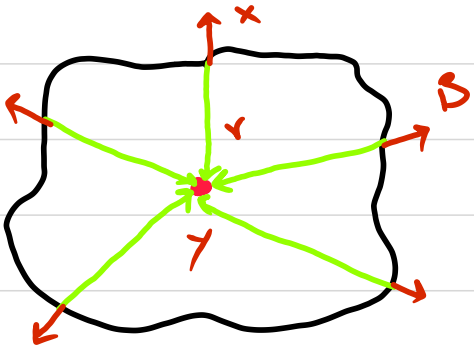
④ Find the local maxima of A & apply a threshold if needed.

The points that were projected into the curves passing through a maximum cell are points belonging to an object of sought shape.

→ ip into curve parmetspace → quantize & Acc isintiatn

→ A counts curves (vot) → local maxima & threshold.

2. **Discretize the Parameter Space**
   - Divide the parameter space into a **grid of M × N cells** (like a 2D histogram).
   - Initialize an **accumulator array** `A(i, j)` of the same dimensions to zero.
3. **Voting Process**
   - Each edge pixel **votes** in the parameter space by increasing the count in every cell that corresponds to a shape it could belong to.
   - In other words: if a pixel lies on multiple potential shapes, it casts votes for all of them.
4. **Find Local Maxima in the Accumulator**
   - Search for cells in `A` with **high vote counts** (local maxima).
   - These represent **shape hypotheses** with strong support from image points.
   - Apply a threshold (if necessary) to filter out weak detections.


x
B
r
y

* Line Detection

→ Detect straight lines in an image using the HT

① Edge detection → get edge pixels
② Initilize 2D accumulator array $A(\theta, \rho)$.

③ For each edge point (x, y):-
→ for each θ in $[-\pi/2, \pi/2]$:
→ compute $\rho = x \cdot \cos\theta + y \cdot \sin\theta$
→ Increment $A(\theta, \rho)$ by 1 → this is a vote for a line with parameter (θ, ρ)

④ After all votes:
→ local maxima in A indicate detected lines in the image.

* Naive Approach

→ GHT extends the classical HT to arbitrary shapes (not necessary defined by equation like a line or circle).

+ Offline phase (Template Modeling)

→ I/P : A template shape

→ steps:

1. Fix a reference point y
2. for each boundary point x on the object:
   - a→ compute gradient direction $\phi(x)$ and discretize it using a step size $\Delta\phi$.
   - b→ compute the vector $r = y - x$
   - c→ store the vector r in an R-table.
     - → Multiple r value may be stored for the same gradient direction.

+ Online Phase (object Detection in image)

→ I/P ⇒ A new target image

→ Step :-

① Apply edge detection to extract edge pixel E.

② Initilize an accumulator array A with same dimensions as the image, all Zeros.

③ For each edge pixel $x \in E$:

ⓐ compute and discretize its gradient direction $\phi(x)$

ⓑ for each vector r_i in the corresponding R-table

- Estimate potential barycenter $y = x + r_i$
- vote in the accumulator $A[y] += 1$

④ Detect local maxima in the accumulator A.

- The maxima are candidates for the object center location.
- You can overly the template at these maxima to find the actual object.